

The ledger sharing technique is a sharing technique in the Peer to Peer network that have similar functionalities to a database, but what it mainly differ from a database is that a database is stored on the server side of the application (centralized), however, a shared ledger is having that database in your device locally. Each and every node connected to the Peet to Peer network will have the exact same ledger stored in all the connected nodes (What makes it shared).

Initialization & Processing

When a new node is successfully connected to the network, initially it must send a `ChainSyncRequest` request to all connected nodes in the network, the request contains the `Action.ChainSyncRequest` action, and it's specifically triggered in the `Chain.outbound_node_connected`. The `ChainSyncRequest` is a message saying that I need the copy of your local ledger.

Or when an existing node has a corrupted ledger, or the entire ledger is missing. In conclusion, the use cases of the ledger sharing are:

1. New node joining the network.
2. An existing node has a corrupted ledger.
3. An existing node loses its local ledger.

```
class Action(Enum):
    ...
    ChainSyncRequest = "CHAIN-SYNC-REQUEST"
    ...

class ChainSyncRequest(BaseModel):
    sender: NodeMetadata
```

And this message must be sent to all connected nodes in the network, each node receives the message, it should trigger the `ChainSync` class to copy its local ledger, verify it and send it back to the node requested it. And the response is a message with the `Action.ChainSyncResponse` action and the actual data is called `ChainSyncResponse`.

```
class Action(Enum):
    ...
    ChainSyncResponse = "Chain-SYNC-RESPONSE"
    ...

class ChainSyncResponse(BaseModel):
```

```
sender: NodeMetadata # The node that received the ChainSync Message.
ledger: list[dict]
```

Class Structure

The *ledger sharing* in our project is called `ChainSync`, it handles all the logic to send and receive a copy of the chain.

```
class ChainSync:
    def __init__(self, ledger: list[dict], network: Node) -> None:
        self.ledger = ledger
        self.network = network # Node object
        self.receivedLedgers: list = [] # Collects ChainSyncResponse.ledger
        self.expectedResponses: int = 0 # Controls when to run the
        self.__bestLeger() function.

    def request(self, message: ChainSyncRequest) -> None:
        self.expectedResponses = len(self.network.all_nodes)
        self.network.send_to_nodes(
            {
                "action": Action.ChainSyncRequest,
                "data": message.model_dump()
            }
        )

    def receive(self, response: ChainSyncResponse) -> None:
        self.receivedLedgers.append(response.ledger)

        if (len(self.receivedLedgers) >= self.expectedResponse):
            bestLedger = self.__chooseBestLedger(self.receivedLedgers)
            self.ledger.replaceChain(bestLedger)
```

The `ChainSync.receive` function is triggered when a node receives a `ChainSyncRequest`, it prepares the copy of the local ledger using the `ChainSync.__copyChain` function, builds the `ChainSyncResponse` response, and sends it to the `ChainSyncRequest.sender` using the `ChainSync.send()` method.

On the other side, when a node receives that `ChainSyncResponse`, it extracts the ledger located at `ChainSyncResponse.ledger`, verifies it using the `ChainValidation` and the `BlockValidation` classes.

However, this is a Peer to Peer network, meaning that we are not receiving one ledger copy, instead we receive a copy of all the ledgers of the connected nodes. This takes us to the next phase: **Ledger Selection**.

Ledger Selection

A ledger is a file that contains a chain of blocks, every block is linked to its previous one, and to share that ledger to a specific node, all the nodes must provide their own copy of their local ledger. Since all the connected nodes should provide their own copy of the ledger, the ledgers may differ, and some will contain more blocks than another, this points the main question:

What ledger to choose? Choosing a ledger is based on two factors:

1. If the ledger provided is valid.
2. It is the longest ledger (most containing blocks).

Example

If a node is given 5 ledgers:

1. $Ledger_1 = 22 \text{ Blocks}$
2. $Ledger_2 = 22 \text{ Blocks}$
3. $Ledger_3 = 23 \text{ Blocks}$
4. $Ledger_4 = 22 \text{ Blocks}$
5. $Ledger_5 = 22 \text{ Blocks}$

We first check if the given ledgers are valid by the `ChainValidation` class defined earlier:

```
# In ChainValidation
def __validateReceivedLedger(self, ledger: list) -> bool:
    """Validates an in-memory chain without reading from disk."""
    block_validator = BlockValidator()
    if not ledger:
        return True
    for block_dict in ledger:
        block = Block.model_validate(block_dict)
        block_validator.validate(block)
    self.__checkChainLinkage(ledger)
    return True
```

Finally if all the ledgers are valid and blocks inside each ledger match each other, we choose the longest chain based on the validity and longest chain. Why? Because the longest ledger is the ledger that has the recent update. This choosing logic is done by the `ChainSync.__chooseBestLedger` function.

```
def __chooseBestLedger(self, ledgers: list[list[dict]]) -> list[dict]:
    validLedgers = [l for l in ledgers if self.__isValid(l)]
```

```

    if not validLedgers:
        return self.ledger.blocks # keep local chain if nothing valid
received

    return max(validLedgers, key=len) # longest valid chain wins

```

Methods

Each private method belongs to a certain logic the `ChainSync` class. **Sending**, **Receiving** and **Verifying**.

Method	Parameters	Logic	Description
<code>__ledgerCopy() -> list[dict]</code>	None	Sending	Returns the copy of the local ledger
<code>__validateBeforeSending(ledger) -> bool</code>	The copy of the ledger	Sending	Verifies the ledger before sending it to the network
<code>__unpack(payload: dict) -> tuple[NodeMetadata, list[dict]]</code>	The payload from the network	Receiving	Takes the payload and extracts the node sent the ledger, and the ledger itself. Returns them as a tuple.
<code>__checkLinkage(ledger: list[dict]) -> bool</code>	The extracted ledger	Verifying	Checks the linkage of the extracted ledger.
<code>__validateReceivedLedger(block: Block) -> bool</code>	The block inside the extracted ledger	Verifying	Checks the block validation, inside the ledger.
<code>__chooseBestLedger(ledger: list[list[dict]]) -> list[dict]</code>	The list of the received ledgers		Chooses the best ledger based on it's validity and it's chain long.

Errors

Logic	Error	Description
Receiving	<code>InvalidChainSyncPayload</code>	Raised when the received <code>ChainSyncRequest</code> contains invalid

Logic	Error	Description
		data.
Verifying	ReceivedLedgerHasInvalidLinkage	Raised when verifying the extracted ledge linkage, and the verification fails.
Verifying	ReceivedLedgerHasInvalidBlock	Raised when an extracted ledger has an invalid block